

1 Làm cho training chạy được (Making Training Work)

PHỤ THUỘC C/12/01-co-hoc-cua-gradient (tích các Jacobian cục bộ — toàn bộ mục này là hệ quả của nó); A/01-toan-toi-thieu (phương sai, giá trị kỳ dị, số điều kiện).

TÓM TẮT

Mục này trả lời câu hỏi trung tâm của cả chương: làm sao đưa *gradient* qua năm mươi tầng mà nó không chết cũng không nổ. Ba nhóm kỹ thuật tưởng như rời rạc — hàm kích hoạt, khởi tạo trọng số, normalization — thực ra nhằm vào đúng một đại lượng duy nhất. Đọc xong bạn nhìn một file cấu hình huấn luyện và nói được từng dòng trong đó đang giữ đại lượng nào ổn định, thay vì chép nguyên recipe của người khác. Nếu chỉ nhớ một điều: adjoint đi ngược qua L tầng là tích của L Jacobian, nên mọi thứ trong mục này chỉ là các cách khác nhau để ép hệ số nhân của tích đó về gần 1.

KIẾN THỨC CẦN CÓ TRƯỚC

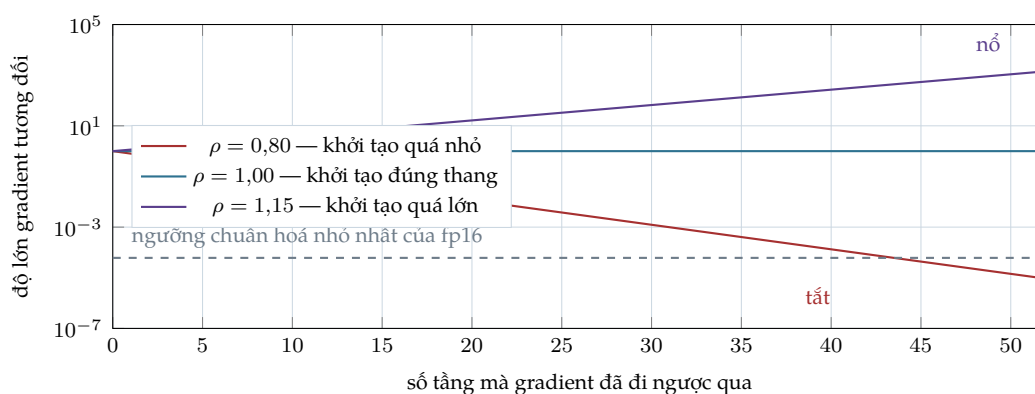
- **tầng (layer)** — một khối phép toán có tham số riêng; “mạng sâu” nghĩa là nhiều tầng nối tiếp, và toàn bộ khó khăn của mục này sinh ra từ chữ “nối tiếp” đó.
- **kích hoạt (activation)** — giá trị đầu ra của một tầng giữa mạng. “Hàm kích hoạt” là phép phi tuyến tạo ra nó, ví dụ ReLU — hai khái niệm khác nhau nhưng tên gần giống.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; đi ngược nó thì mất mát giảm. Gradient “tiêu biến” nghĩa là nó nhỏ tới mức tầng đó không học được nữa.
- **phương sai (variance)** — đại lượng đo độ tản mát của một tập số quanh trung bình của chúng; cả lập luận khởi tạo trọng số trong mục này chỉ là theo dõi phương sai đi qua từng tầng.
- **số điều kiện (condition number)** — tỉ số giữa độ cong lớn nhất và nhỏ nhất của mặt mất mát; càng lớn thì đường đồng mức càng dẹt và thuật toán tối ưu càng đi zigzag.
- **batch (lô)** — nhóm mẫu đi qua mạng cùng lúc. Với BatchNorm, kích thước lô không chỉ là chuyện tốc độ mà còn thay đổi *chính hàm số* mà mạng biểu diễn.
- **chế độ huấn luyện và suy luận (train / eval mode)** — cùng một mô hình hành xử khác nhau ở hai chế độ: BatchNorm và dropout dùng thống kê batch khi huấn luyện, dùng thống kê tích lũy khi suy luận. Quên chuyển chế độ là bug kinh điển ở cuối mục.
- **bias — phân biệt hai nghĩa** — trong $Wx + b$, b là tham số cộng của một tầng, và “đẩy bias xuống rất âm” ở phần ReLU là nói về nó. Nghĩa thống kê — độ lệch hệ thống của một bộ ước lượng — không dùng trong mục này.

A. Đặt vấn đề

Gradient đã tính đúng, kiểm tra bằng số đã qua, code chạy không lỗi. Vậy mà loss vẫn nằm im ở mức phỏng đoán ngẫu nhiên, hoặc bay lên NaN ở bước thứ ba trăm. Đây là trạng thái mà mọi người từng huấn luyện mạng sâu đều đã sống qua, và nó không phải một lỗi lập trình. Nó là biểu hiện của nút thắt lịch sử lớn nhất của ngành: *vanishing and exploding gradient* — gradient tắt dần hoặc nổ tung khi đi ngược qua nhiều tầng. Giữa đầu thập niên 1990 và khoảng 2012, người ta biết cách viết mạng sâu và biết cách tính gradient của nó. Chỉ có điều không ai huấn luyện được nó. Gần như mọi “trick” trong một file cấu hình huấn luyện hiện đại tồn tại vì lý do đó.

Cơ chế thì rất gọn, và nó suy trực tiếp ra từ mục trước. Adjoint đi ngược qua L tầng là một tích của L Jacobian cục bộ. Nếu giá trị kỳ dị điển hình của mỗi Jacobian là ρ , thì độ lớn gradient ở tầng đầu tiên tỉ lệ với ρ^L . Đây là hàm mũ, nên không có vùng an toàn rộng: với $L = 50$, $\rho = 0,9$ cho $0,9^{50} \approx 5 \cdot 10^{-3}$ và $\rho = 1,1$ cho $1,1^{50} \approx 117$. Một sai lệch 10 phần trăm ở mỗi tầng biến thành hai bậc độ lớn ở toàn mạng theo hướng này và hai bậc theo hướng kia.

↪ **hệ quả của** C/12/01 — tích các Jacobian là kết quả trực tiếp của việc adjoint đi ngược qua từng tầng một



Hình 1. Vì sao không có vùng an toàn rộng cho khởi tạo. Ba đường chỉ khác nhau ở hệ số ρ của một tầng — lệch 20 phần trăm theo hai chiều — nhưng sau 50 tầng chúng cách nhau tám bậc độ lớn, vì lũy thừa thắng mọi thứ khác. Đường $\rho = 0,80$ chui xuống dưới ngưỡng biểu diễn của fp16 từ khoảng tầng thứ 40, tức các tầng đầu mạng không nhận được tín hiệu nào cả. Đây là đồ thị của một mô hình đơn giản hoá (mọi Jacobian có cùng phổ, độc lập nhau) nên nó cho thấy *chiều* và *tốc độ* của hiện tượng, không dự đoán giá trị cụ thể.

Hình 1 là toàn bộ lý do ba mục tiếp theo tồn tại: khoảng cách giữa “huấn luyện được” và “không huấn luyện được” chỉ là vài phần trăm ở hệ số của một tầng.

Do đó bài toán không phải “làm cho ρ tốt” mà là “làm cho ρ bằng 1 và giữ nó ở đó suốt quá trình huấn luyện”.

Có đúng ba hướng tấn công vào cùng con số ρ đó, và toàn bộ mục này là ba hướng ấy.

- **Sửa hệ số danh nghĩa** — chọn hàm kích hoạt có đạo hàm không nhỏ đi (mục C).
- **Đặt điểm làm việc ban đầu** — chọn khởi tạo sao cho phương sai tín hiệu giữ nguyên qua mỗi tầng (mục D).
- **Hồi tiếp trong lúc chạy** — chèn phép chuẩn hoá ở giữa mạng để ép lại phân bố mỗi khi nó trôi (mục E).

Nhưng trước cả ba, có một việc rẻ hơn nhiều phải làm trước ở đầu vào. Giá trị cụ thể của cả mục: bạn sẽ đọc được một cấu hình huấn luyện lạ và chỉ ra chính xác dòng nào đang giữ ρ gần 1, do đó biết dòng nào có thể bỏ và dòng nào không.

TRỰC GIÁC

Coi mạng sâu như một chuỗi năm mươi bộ khuếch đại nối tiếp nhau, mỗi bộ có hệ số khuếch đại danh nghĩa là 1 nhưng lệch chuẩn vài phần trăm. Không ai chỉnh từng bộ để tổng thể đúng bằng 1 — sai số nhân dồn sẽ thắng. Cái người ta làm trong điện tử là mắc hồi tiếp âm quanh mỗi tầng để hệ số của tầng đó bị ép về giá trị mong muốn bất kể linh kiện trôi. Ba hướng trong mục này là ba kiểu hồi tiếp khác nhau lên cùng một đại lượng: hàm kích hoạt sửa hệ số danh nghĩa, khởi tạo đặt điểm làm việc ban đầu, normalization là vòng hồi tiếp thật sự vì nó đo phân bố đang chạy rồi kéo nó về.

B. Chuẩn hoá đầu vào (input normalization): việc rẻ nhất, làm trước tiên

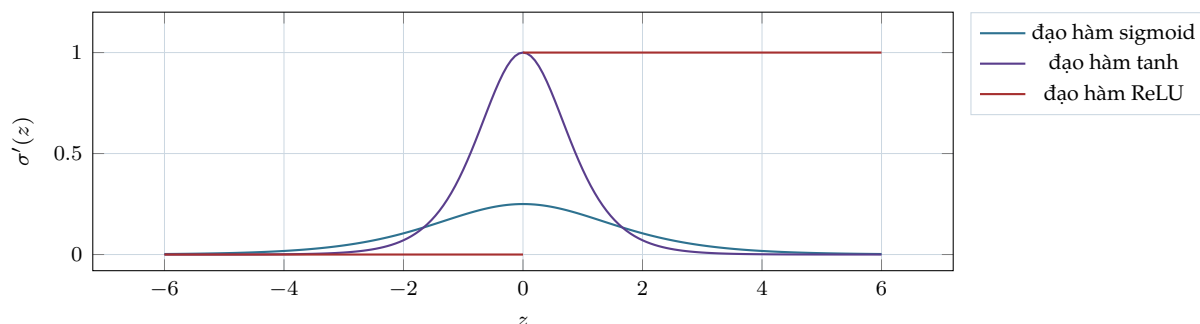
Trước khi nói về chiều sâu, hãy nhìn một tầng tuyến tính duy nhất. Hessian của mất mát bình phương theo trọng số của tầng đó tỉ lệ với $\mathbb{E}[xx^T]$, tức ma trận moment bậc hai của đầu vào. Giả sử một đặc trưng nằm trong $[0, 1]$ còn đặc trưng kia trong $[0, 1000]$. Hai trị riêng của Hessian khi đó lệch nhau khoảng 10^6 , và đường đồng mức của mất mát là những ellipse cực dẹt. Gradient descent trên một ellipse dẹt đi zigzag. Bước học phải đủ nhỏ để không phân kỳ theo trục dọc,

nên nó nhỏ đến vô lý theo trực giác. Tốc độ hội tụ khi đó do **số điều kiện** (*condition number*) κ chi phối, theo hệ số $\left(\frac{\kappa-1}{\kappa+1}\right)^t$. Với $\kappa = 1000$, hệ số đó là 0,998 mỗi bước — cần khoảng 350 bước chỉ để giảm sai số đi một nửa.

Trung bình và chia độ lệch chuẩn theo từng kênh đưa κ từ 10^6 về vài đơn vị với chi phí gần bằng không. Đây là lý do vì sao nó là dòng đầu tiên của mọi pipeline, chứ không phải vì một lý do thẩm mỹ nào. Bước xa hơn về lý thuyết là *whitening* bằng PCA, đưa κ về đúng 1; nhưng với ảnh thì nó gần như không bao giờ được dùng, vì hai lý do có thể tính ra được. Thứ nhất là chi phí. Một ảnh $3 \times 224 \times 224$ có 150 528 chiều, nên ma trận hiệp phương sai của nó có $150\,528^2 \approx 2,27 \cdot 10^{10}$ phần tử. Riêng việc lưu nó đã tốn khoảng 90 GB ở fp32, chưa nói tới phân rã. Thứ hai, và quan trọng hơn, whitening trộn mọi pixel với mọi pixel nên nó *phá cấu trúc không gian*: sau khi whiten, hai pixel cạnh nhau không còn cạnh nhau theo nghĩa thống kê, và toàn bộ giả định **chia sẻ trọng số** (*weight sharing*) của *convolution* mất cơ sở. Đây là một ví dụ sạch về nguyên lý “một phép biến đổi tối ưu theo một tiêu chí có thể phá huỷ đúng cấu trúc mà phần còn lại của hệ thống đang dựa vào”.

C. Hướng 1 — hàm kích hoạt (kích hoạt function), đọc như một chuỗi sửa lỗi

Hàm kích hoạt (*activation function*) đóng góp trực tiếp vào ρ : Jacobian của nó là một ma trận đường chéo chứa $\sigma'(z)$, nên nếu σ' nhỏ thì ρ nhỏ. Sigmoid có σ' cực đại bằng 0,25 tại $z = 0$ và tiến nhanh về 0 ở hai đầu. Ngay cả ở điều kiện tốt nhất, mười tầng sigmoid nhân vào nhau cho hệ số $0,25^{10} \approx 10^{-6}$; ở điều kiện thực tế, nơi phần lớn nơ-ron bão hoà, con số còn tệ hơn nhiều. Một phép nhân hai dòng như vậy giải thích vì sao mạng sâu không huấn luyện được trước 2010, thuyết phục hơn bất kỳ lời giải thích định tính nào.



Hình 2. Đạo hàm của ba hàm kích hoạt — đây là đại lượng thật sự đi vào tích các Jacobian, chứ không phải bản thân hàm. Đỉnh của đạo hàm sigmoid chỉ là 0,25 và tụt nhanh về 0 ở hai đầu; đạo hàm tanh đạt 1 nhưng cũng bão hoà; ReLU giữ đúng 1 trên toàn nửa dương và trả về 0 trên nửa âm, đối tính bão hoà lấy nguy cơ nơ-ron chết.

Hình 2 vẽ đúng đại lượng cần nhìn: không phải hình dạng hàm kích hoạt mà là hình dạng *đạo hàm* của nó, vì chỉ đạo hàm mới đi vào tích Jacobian. Bảng 1 đọc tiếp bức tranh đó theo trục thời gian.

Bảng 1. Chuỗi hàm kích hoạt đọc như một chuỗi sửa lỗi. Cột cuối là cột đáng nhớ: mỗi bước sửa được một khiếm khuyết và để lại một khiếm khuyết mới.

Hàm	Cơ chế	Sửa được gì	Vấn hòng ở đâu
Sigmoid	Ép về $(0, 1)$	— (khởi điểm lịch sử)	Bão hoà hai đầu giết gradient; đầu ra không quanh 0 nên gradient của cả tầng cùng dấu
Tanh	Ép về $(-1, 1)$	Đầu ra quanh 0, hết hiện tượng cùng dấu	Vẫn bão hoà, $\sigma' \leq 1$
ReLU	$\max(0, x)$	Không bão hoà phía dương ($\sigma' = 1$), tính cực rẻ	Nơ-ron chết vĩnh viễn nếu rơi hẳn sang phía âm; không quanh 0
Leaky ReLU, PReLU, ELU	Cho phía âm một độ dốc nhỏ	Nơ-ron chết	Thêm siêu tham số; lợi ích không ổn định giữa các bài toán
GELU, SiLU	Cổng mượt theo giá trị đầu vào	Đạo hàm liên tục, hợp với transformer và lịch học dài	Đắt hơn ReLU vài chục phần trăm; khác biệt nhỏ trên CNN

Bảng 1 đọc theo chiều dọc thì thấy rõ đây không phải một danh sách lựa chọn ngang hàng mà là một chuỗi nhân quả. Điều đáng chú ý là bước tiến lớn nhất — từ tanh sang ReLU — không đến từ một hàm “tốt hơn” theo nghĩa toán học mà từ việc *bỏ đi* tính bão hoà. Đáng chú ý không kém là ReLU tạo ra một chế độ hỏng hoàn toàn mới không tồn tại trước đó: một nơ-ron nhận bước cập nhật lớn đẩy bias xuống rất âm sẽ cho 0 trên mọi dữ liệu, do đó gradient của nó bằng 0 mãi mãi, và nó chết không hồi phục. Tỷ lệ nơ-ron chết (*dead ReLU*) vì thế là một chỉ báo sức khoẻ đáng đo, và ta sẽ gặp lại nó ở mục sau.

D. Hướng 2 — khởi tạo trọng số (weight initialization), hay toán đơn giản giải bài toán thật

Lập luận phương sai là ví dụ đẹp nhất trong ngành về việc vài dòng đại số giải quyết một vấn đề đã chặn cả một lĩnh vực. Xét $y = Wx$ với n đầu vào, các w_{ij} độc lập cùng phân bố trung bình 0, và x độc lập với W . Khi đó

$$\text{Var}(y_j) = n \text{Var}(w) \text{Var}(x).$$

Muốn **phương sai** (*variance*) tín hiệu không đổi qua mỗi tầng, cần $n \text{Var}(w) = 1$, tức $\text{Var}(w) = 1/n_{\text{in}}$. Nhưng lượt ngược cũng phải giữ phương sai, và ở đó vai trò của n_{in} và n_{out} đổi chỗ; không thoả cả hai cùng lúc được nên Xavier/Glorot chọn trung hoà $\text{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}})$ [1]. Đó là lời giải cho mạng tanh.

Với ReLU, lập luận trên thiếu một hệ số. ReLU xoá một nửa số đầu ra về 0, nên phương sai sau kích hoạt chỉ còn một nửa phương sai trước nó; hệ số mỗi tầng là $1/2$ chứ không phải 1. Sau 30 tầng, tín hiệu bị nhân với $2^{-30} \approx 10^{-9}$ — tức là tắt hẳn, dù bạn đã “khởi tạo đúng” theo Xavier. Sửa lại thì hiển nhiên: nhân đôi phương sai, $\text{Var}(w) = 2/n_{\text{in}}$. Đó là khởi tạo He/Kaiming [2], và chính hệ số 2 đó là thứ cho phép huấn luyện mạng ReLU ba mươi tầng lần đầu tiên. Hãy để ý mạch lập luận: một giả định (“ReLU không đổi phương sai”) bị vi phạm, hậu quả được lượng hoá ra một con số, và lời sửa là một hằng số. Đây là dạng suy luận mà bạn nên tái tạo được, chứ không phải hai công thức để nhớ.

↔ **tiền đề** A/01 — toàn bộ lập luận chỉ dùng một tính chất của phương sai: phương sai của tổng các biến độc lập bằng tổng phương sai

E. Hướng 3 — normalization, vòng hồi tiếp thật sự

Hai hướng trên chỉ đặt điều kiện *ban đầu*. Sau vài nghìn bước, trọng số đã đổi và phân bố kích hoạt trôi khỏi chỗ được thiết kế. **Chuẩn hoá** (*normalization*) đo phân bố đang chạy rồi ép nó về trung bình 0 và độ lệch chuẩn 1. Sau đó nó trả lại quyền chính cho mạng, qua hai tham số học được γ và β . Bốn biến thể khác nhau ở đúng một điểm: *lấy trung bình trên trục nào*.

Bảng 2. Bốn biến thể normalization. Chúng chỉ khác nhau ở tập chỉ số dùng để tính thống kê, và chính lựa chọn đó quyết định chúng hỏng khi nào.

Loại	Trung bình theo	Sinh ra để giải quyết	Hỏng khi nào
BatchNorm [3]	Batch $\times$ không gian, riêng từng kênh	Ổn định phân bố giữa mạng, cho phép dùng learning rate lớn hơn hẳn	Batch nhỏ (thống kê nhiễu); lệch train/eval; phải đồng bộ giữa các GPU
LayerNorm [4]	Toàn bộ đặc trưng của <i>một</i> mẫu	Batch nhỏ hoặc bằng 1; chuỗi độ dài thay đổi; không phụ thuộc mẫu khác	Trên CNN thì giác thường kém BN; trộn chung các kênh có ý nghĩa rất khác nhau
GroupNorm [5]	Nhóm kênh của một mẫu	Detection/segmentation với batch một tới hai ảnh mỗi GPU	Thêm siêu tham số số nhóm; không có lợi thế nào khi batch đủ lớn
InstanceNorm	Mỗi kênh của mỗi mẫu	Style transfer: xoá thống kê style khỏi đặc trưng	Ở bài toán khác, thứ bị xoá lại là thông tin có ích (độ sáng, tương phản)

Bảng 2 nên đọc cùng một nhận xét về mặt cơ chế: câu chuyện gốc của BatchNorm là “giảm internal covariate shift”, nhưng lời giải thích đó đã bị chất vấn nghiêm túc. Santurkar và cộng sự bơm nhiễu để *tăng* covariate shift một cách cố ý, và BN vẫn hoạt động tốt. Họ đề xuất tác dụng thật là làm mất mát mát trơn hơn. Gradient bớt thay đổi đột ngột, nên bước đi dài hơn mà không phân kỳ [6]. Đây là một trường hợp đáng nhớ khi đọc paper. Kỹ thuật đúng, nhưng lời giải thích đi kèm nó lúc ra đời có thể sai. Học thuộc lời giải thích thay vì cơ chế thì bạn sẽ suy luận sai ở tình huống kế tiếp.

Hệ quả kỹ thuật quan trọng nhất của BatchNorm là nó *ghép các mẫu trong batch lại với nhau*. Ta đã thấy điều đó ở công thức backward trong mục trước: gradient của một mẫu phụ thuộc mọi mẫu cùng batch. Từ đó suy ra gần như mọi chế độ hỏng của nó. Batch nhỏ thì thống kê nhiễu, nên gradient nhiễu. Lúc suy luận không có batch nên phải dùng thống kê chạy tích lũy trong huấn luyện, và bất kỳ chênh lệch nào giữa hai chế độ đều thành sai lệch hệ thống. Trong huấn luyện phân tán, mỗi tiến trình tính thống kê riêng nên mô hình thực chất khác nhau ở mỗi GPU trừ khi bạn dùng SyncBN. LayerNorm và GroupNorm né toàn bộ họ vấn đề này bằng cách không dùng tới trục batch — và đó là toàn bộ lý do transformer dùng LayerNorm, chứ không phải vì LayerNorm “hiện đại hơn”.

↪ **ràng buộc bởi** C/14 — detection chạy một tới hai ảnh mỗi GPU, nên chính ràng buộc batch nhỏ ở đó quyết định lựa chọn GroupNorm

(1) conv → chuẩn hoá → hàm kích hoạt — thứ tự mặc định



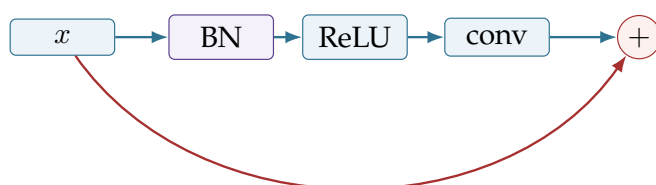
Đầu vào ReLU có trung bình 0, nên khoảng một nửa số nơ-ron hoạt động ở mọi batch. Đây là cấu hình mà lập luận phương sai của He giả định.

(2) conv → hàm kích hoạt → chuẩn hoá



BN phải chuẩn hoá một phân bố *không âm*, nên β phải gánh việc dời tâm và γ làm việc trên một phổ hẹp hơn. Chạy được, nhưng lãng phí hai tham số.

(3) tiền kích hoạt trong khối residual: BN → ReLU → conv



Gradient từ đầu ra về x đi qua một đường không bị chuẩn hoá chặn, nên thành phần bằng 1 trong Jacobian được giữ nguyên qua rất nhiều khối.

đường tắt *sạch*: không có phép toán nào chen vào

Hình 3. Ba cách đặt phép chuẩn hoá quanh hàm kích hoạt, và hệ quả lên luồng gradient. Điều hình này nói mà câu văn khó nói: vị trí của khối BN không phải chi tiết thẩm mỹ. Nó quyết định phân bố mà ReLU nhìn thấy, tức hàng 1 so với hàng 2. Nó cũng quyết định đường tắt có sạch hay không, tức hàng 3. Ở hàng 3, mọi phép toán đã bị đẩy vào *nhánh residual*, nên đường cộng từ x tới điểm $+$ hoàn toàn trống. Đó là lý do biến thể tiền kích hoạt huấn luyện được ở độ sâu lớn hơn hẳn.

Hình 3 cũng là chỗ ba hướng của mục này gặp nhau trên một sơ đồ duy nhất: hàng 1 nói về quan hệ giữa chuẩn hoá và hàm kích hoạt, hàng 3 nói về quan hệ giữa chuẩn hoá và kết nối tắt.

Còn một hướng thứ tư mà mục này cố tình không kể: **kết nối tắt** (*residual connection*). Nó tấn công đúng cùng đại lượng ρ , nhưng bằng cách thêm một đường cộng để Jacobian có dạng $I + \partial F / \partial x$ — luôn có một thành phần bằng 1 không thể triệt tiêu. Vì nó là một lựa chọn *kiến trúc* chứ không phải một thủ thuật huấn luyện, nó thuộc về C/13/01-dong-cnn; nhưng khi đọc chương đó, hãy nhớ nó là thành viên thứ tư của đúng gia đình này.

↔ cùng nguyên lý với C/13/01 — kết nối tắt giữ tích Jacobian gần 1 bằng một đường cộng, tức cùng mục tiêu với ba hướng ở đây nhưng thực hiện ở tầng kiến trúc

F. Giới hạn và trường hợp hỏng

Ba hướng trên không cộng dồn vô hạn, và chúng có những chỗ giao thoa gây bất ngờ.

Khởi tạo đúng chỉ đúng ở bước 0. He/Kaiming giả định trọng số độc lập cùng phân bố và đầu vào đã chuẩn hoá; sau vài nghìn bước cả hai giả định đều sai, nên khởi tạo không thay được normalization. Ngược lại, ở các mạng rất sâu có kết nối tắt, khởi tạo quan trọng theo một kiểu khác. Nhánh residual cần được khởi tạo nhỏ hoặc bằng 0, để mạng bắt đầu gần với ánh xạ đồng nhất. Bỏ chi tiết đó đi thì mạng sâu vẫn khó huấn luyện, dù đã có đủ BN.

Normalization che giấu lỗi. Một mạng có BN ở khắp nơi vẫn huấn luyện được dù khởi tạo sai, dù đầu vào chưa chuẩn hoá, dù learning rate lệch một bậc — chỉ là kém hơn đáng kể. Đây là tin tốt cho tiến độ và tin xấu cho việc học chẩn đoán, vì nó xoá mất các triệu chứng mà lẽ ra bạn phải học đọc.

Cuối cùng, các con số như ρ^L hay 2^{-30} trong mục này là ước lượng bậc độ lớn dựa trên giả định các Jacobian độc lập và có phổ đều. Giả định đó thô: trong thực tế các Jacobian tương quan với nhau, và phổ của chúng không đều. Chúng dùng để cho thấy *hàm mũ thắng mọi thứ khác*, chứ không dùng để dự đoán một giá trị cụ thể.

CẠM BẤY

Bug kinh điển của cả nhóm này: quên gọi `model.eval()` khi đánh giá. BatchNorm khi đó dùng thống kê của chính batch kiểm thử thay vì thống kê chạy. Kết quả vì thế phụ thuộc vào việc bạn xếp mẫu nào chung batch. Tệ nhất là nó thường *tốt hơn* thực tế, vì mô hình được nhìn trộm thống kê của tập kiểm thử. Triệu chứng nhận dạng: điểm số đôi khi bạn đổi batch size lúc đánh giá, hoặc điểm số tụt hẳn khi triển khai với batch bằng 1. Cùng một dòng lệnh cũng tắt dropout, nên hai lỗi này luôn đi cùng nhau.

CẦN NHỚ

Cả ba hướng — hàm kích hoạt, khởi tạo, normalization — chỉ nhắm vào đúng một đại lượng: hệ số nhân của tích các Jacobian qua các tầng. Mọi câu hỏi dạng “kỹ thuật X có cần thiết không” nên được dịch lại thành “trong cấu hình của tôi, cái gì đang giữ hệ số đó gần 1”. [T1]

G. Kết nối

Mục này lấy đầu vào từ C/12/01-co-hoc-cua-gradient: toàn bộ lập luận chỉ là hệ quả số học của việc adjoint là một tích Jacobian. Nó chuyển tiếp sang C/12/03-toi-uu-hoa. Ở đó ta giả định gradient đã lành mạnh, rồi hỏi tiếp nên đi bước dài bao nhiêu. Warmup sẽ xuất hiện ngay tại đó, như hệ quả trực tiếp của việc thống kê BN chưa ổn định ở những bước đầu. Thành viên thứ tư của gia đình, kết nối tất, nằm ở C/13/01-dong-cnn. Lựa chọn giữa BatchNorm và GroupNorm gắn chặt với ràng buộc batch size trong detection, nên nó được nhắc lại ở C/14-bai-toan-cv-loi; còn chi phí bộ nhớ của việc lưu $\hat{x}$ cho lượt ngược thuộc về B/11-gioi-han-tinh-toan.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một sai lệch 10 phần trăm ở hệ số mỗi tầng lại là vấn đề nghiêm trọng, trong khi sai lệch 10 phần trăm ở learning rate thì không?
2. Whitening đưa số điều kiện về 1, tức tối ưu theo tiêu chí ở đầu mục. Vậy vì sao gần như không ai whiten ảnh?
3. Hệ số 2 trong khởi tạo He đến từ đâu, và nếu bạn thay ReLU bằng GELU thì lập luận đó thay đổi thế nào?
4. BatchNorm hỏng theo ba kiểu khác nhau; cả ba đều suy ra từ một tính chất duy nhất của nó. Tính chất đó là gì?
5. Vì sao lời giải thích “internal covariate shift” bị nghi ngờ, và vì sao việc học thuộc một lời giải thích sai lại nguy hiểm hơn là không có lời giải thích nào?
6. Bạn thay BatchNorm bằng LayerNorm trong một CNN thị giác và kết quả tệ đi — cách giải thích của bạn là gì, và phép đo nào kiểm chứng được nó?

Nguồn

- [1] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [2] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [3] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [4] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [5] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [6] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.